

pytest-gpu-proof: Enabling Cloud-CPU Continuous Integration for GPU Code with Local GPU Attestation

Brian Plancher¹

Abstract—GPU acceleration is now routine across robotics, but cloud-hosted GPU continuous integration (CI) runners are expensive, resulting in severe under-testing of GPU-accelerated code. We present **pytest-gpu-proof**, an open-source **pytest** plugin offering a practical middle ground. Tests can be run on a local machine, signed with a receipt of exactly what ran and what it produced, and integrated into standard CPU CI workflows (e.g., GitHub Actions). The tool is open source and on PyPI, and we are actively integrating it across our lab’s software stack.

I. INTRODUCTION

GPU acceleration is now routine across robotics and the trend is only accelerating. In fact, papers with “GPU” in the title or abstract grew from 25 at IROS and ICRA 2023 combined to 89 at IROS 2025 and ICRA 2026 combined. This includes work developed by industry and academia on topics from across the robotics stack, including: simulation, computer vision, estimation, mapping, planning, control, end-to-end learning and more [1], [2]. This is all occurring at a time when the number of paper submissions across academia is rapidly rising and AI coding tools have not only supported this trend but also led to a proliferation of poorly tested code that is hard to maintain [3]–[6].

This underscores that academia, and software engineering more broadly, needs tools that can help low-resourced developers (e.g., common in small academic labs) maintain and test their code through industry best practices like continuous integration (CI) [7], [8].

At the same time, while popular cloud-based code repositories like GitHub support CI workflows [7], [8] that can run popular testing suites (e.g., **pytest** [9]), running GPU programs in such environments often requires the use of costly advanced features [10]. For

¹Brian Plancher is with Dartmouth College, Hanover, NH, USA. (Email: plancher@dartmouth.edu)

This project was supported by the National Science Foundation (Award 2411369) and the Toyota Research Institute. Any opinions, findings, conclusions, or recommendations expressed in this material are those of the author and do not necessarily reflect those of the funding organizations.

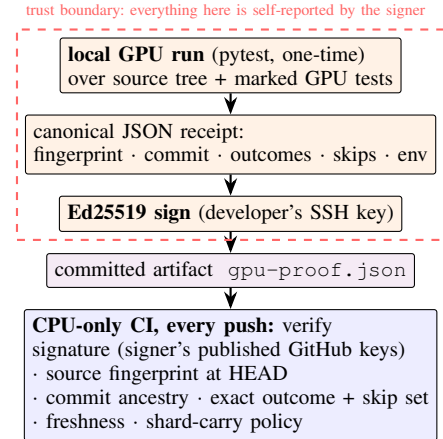


Fig. 1. The (possibly expensive or time consuming) GPU run is local and is signed. Every following (cloud-based) CI run verifies the signed attestation in seconds and fails on any drift between the attested run and the current tree.

all but the largest corporations, cloud-based GPU CI is thus generally avoided. This means that open-source, GPU-accelerated robotics academic software is often severely under-tested despite the fact that this code often contains difficult to detect errors (e.g., data-dependent race conditions [11], [12]).

We address this issue with **pytest-gpu-proof**, a plugin for **pytest** that enables GPU tests to be run locally, cryptographically signed to provide attestation of exactly what ran and what it produced, and integrated into CPU-based CI workflows on popular repositories like GitHub. Our plugin is lightweight and easy to install as it only requires **pytest** and an SSH key, and its own CI runs at 100% test coverage. We hope this tool helps enable the explosion of GPU-based robotics software to be better maintained and more reliable. We have released our tool open source under an MIT license at <https://github.com/A2R-Lab/pytest-gpu-proof>, with version 0.4.0 on PyPI (`pip install pytest-gpu-proof`).

II. PLUGIN DESIGN AND OPERATION

pytest-gpu-proof operation can be summed up in one clause: run locally, sign, and verify everywhere.

As shown in Fig. 1, tests that need a GPU are selected with a `pytest` marker and run locally. We then record the collected test set, per-test outcomes and durations, expected skips, git commit and tree state, the environment and reported GPU, and a SHA-256 fingerprint over the configured source scope (by default every tracked file). This record is serialized as a JSON file and signed with the developer’s existing Ed25519 SSH key (verified later against published GitHub keys). The receipt is then committed with the code, and the CI (local or cloud) performs one quick, CPU-only step to verify the receipt. That is, verification recomputes the source fingerprint at HEAD, requires the recorded commit to be the current commit or an ancestor, requires every recorded test to have passed, compares the *exact* expected-skip set, and applies freshness and clean-tree policies to ensure no edits were made after the signed test receipt.

Beyond simple GPU test markers, our plugin includes a comparison feature that pairs a GPU kernel under test with a (user provided) CPU or GPU reference implementation and records the outcome in the receipt.

`pytest-gpu-proof` supports a variety of different repository policies through the use of a small declarative policy file that e.g., pins the required fingerprint scope, maximum receipt age, recording mode, and an optional test manifest. To ensure that a misspelling cannot silently weaken the policy, unknown policy fields fail verification. For large suites, receipts can be recorded in *shards* on different machines and merged, with a carry-forward policy that lets shards whose fingerprinted inputs are unchanged ride along without rerunning. Policies can also differentiate between acceptable CI for a single commit as compared to a release (e.g., requiring a full test set run before a release). Finally, policies can specify whether the signer policy is *open*, which enables a valid receipt from any GitHub user (e.g., to support contributor-signed pull requests), or *restricted*, which allowlists specific users for added security (e.g., around releases).

III. WHAT A RECEIPT DOES AND DOES NOT PROVE

With `pytest-gpu-proof`, the claim “the GPU tests pass” is pinned to an exact source tree, commit, test list, outcome set, and date, and is re-checked on every push, so accidental staleness (the dominant failure mode in practice) is caught immediately. However, a receipt remains only an *attestation by a keyholder*, not cryptographic proof of GPU execution as a dishonest signer can sign anything. That being said, this is

the same trust model as a signed git tag or release artifact [13], and a *restricted* signing policy limits that trust to a chosen set of developers.

IV. CONCLUSION AND FUTURE WORK

`pytest-gpu-proof` enables CI of GPU projects while keeping expensive GPU compute on local hardware. We hope the workshop discussion will help refine the policy vocabulary and we welcome adversarial feedback on our plugin design and trust model.

ACKNOWLEDGMENTS

We used LLM tools, including Codex and Claude Code, to assist with software implementation, proof-reading, and review. All final text and references were edited and reviewed by humans.

REFERENCES

- [1] IEEE, “Ieee xplore digital library,” <https://ieeexplore.ieee.org/>, 2026.
- [2] IEEE RAS, “Icra 2026 technical program,” <https://ras.papercept.net/conferences/conferences/ICRA26/program/>, 2026.
- [3] A. Billard, R. Detry, N. Figueroa, M. Foriest, D. Lee, and K. Yao, “Surviving the paper deluge: A one-year study in learning from demonstration [science and technology watch],” *IEEE Robotics & Automation Magazine*, vol. 33, no. 2, pp. 205–211, 2026.
- [4] Z. Wang, C. Wang, Y. Yu, S. Wang, F. Xiong, W. Zhang, *et al.*, “Maintaincoder: Maintainable code generation under dynamic requirements,” *Advances in Neural Information Processing Systems*, vol. 38, pp. 15 399–15 436, 2026.
- [5] A. Ghammam and M. Almukhtar, “Ai builds, we analyze: An empirical study of ai-generated build code quality,” in *Proceedings of the 23rd International Conference on Mining Software Repositories*, 2026, pp. 727–731.
- [6] R. T. Lange, Q. Sun, A. Prasad, M. Faldor, Y. Tang, and D. Ha, “Towards robust agentic CUDA kernel benchmarking, verification, and optimization,” 2025, arXiv:2509.14279.
- [7] P. M. Duvall, S. Matyas, and A. Glover, *Continuous integration: improving software quality and reducing risk*. Pearson Education, 2007.
- [8] M. Shahin, M. A. Babar, and L. Zhu, “Continuous integration, delivery and deployment: a systematic review on approaches, tools, challenges and practices,” *IEEE access*, vol. 5, pp. 3909–3943, 2017.
- [9] pytest, “pytest: helps you write better programs,” <https://docs.pytest.org/en/stable/>, 2026.
- [10] GitHub, “Continuous integration,” <https://docs.github.com/en/actions/get-started/continuous-integration>, 2026.
- [11] Y. Yu, T. Rodeheffer, and W. Chen, “Racetrack: efficient detection of data race conditions via adaptive tracking,” in *Proceedings of the twentieth ACM symposium on Operating systems principles*, 2005, pp. 221–234.
- [12] A. Betts, N. Chong, A. F. Donaldson, S. Qadeer, and P. Thomson, “GPUVerify: A verifier for GPU kernels,” in *Proc. OOPSLA*, 2012.
- [13] Z. Newman, J. S. Meyers, and S. Torres-Arias, “Sigstore: Software signing for everybody,” in *Proc. ACM Conference on Computer and Communications Security (CCS)*, 2022.